

Mini Project 3 -Real-Time Book Recommendation System

DSAI 427 Big Data Analytics

Mohamed Hozien

Ziad Moataz

May 2026

1 Executive Summary

We built an end-to-end big-data pipeline that learns book preferences from the Kindle Reviews dataset using Spark MLlib's ALS, streams new interactions through Kafka, computes 30-second sliding-window analytics with Spark Structured Streaming, and serves hybrid Top-5 recommendations dynamically. The system is augmented with a Streamlit dashboard and meets a sub-five-second end-to-end latency target.

Domain: Books (Kindle Reviews, ~982K ratings).

Focus: Real-Time Intelligence (trending detection + alerts) with a slice of System Optimization for the latency bonus.

Bonuses pursued: Streamlit dashboard (+2), latency <5 s (+1).

2 System Architecture

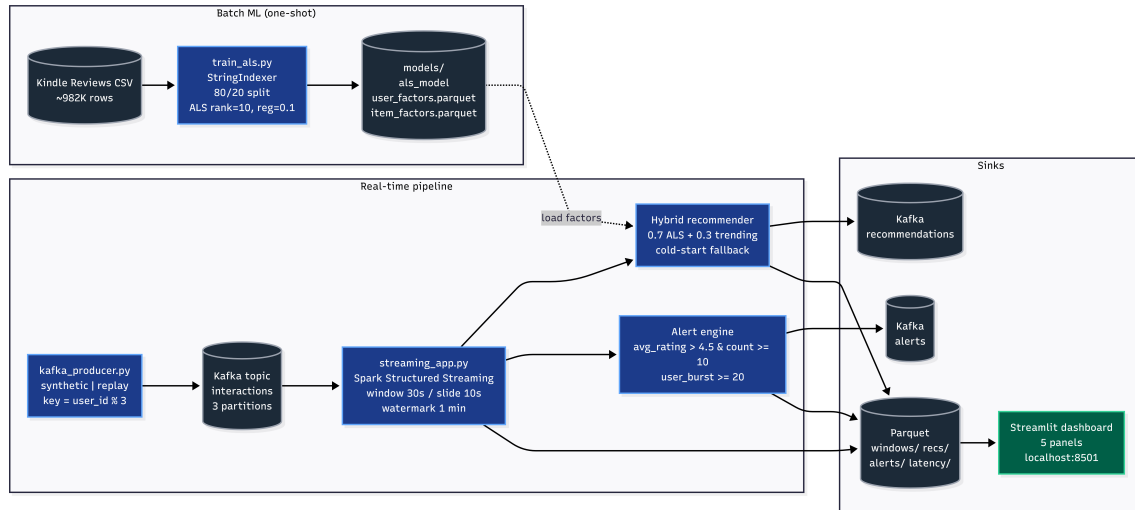


Figure 1: End-to-end pipeline: batch ALS training feeds factor parquets into the streaming app; Kafka interactions (3 partitions, key=user_id%3) drives Spark Structured Streaming, which emits hybrid recommendations and alerts to Kafka topics and parquet sinks; the Streamlit dashboard polls the sinks.

3 Dataset and Justification

The dataset is the Kindle Reviews release on Kaggle ([bharadwaj6/kindle-reviews](https://www.kaggle.com/bharadwaj6/kindle-reviews)). It contains ~982K rows with columns `reviewerID`, `asin`, `overall`, `unixReviewTime`, mapping cleanly

to the required (`user_id`, `item_id`, `rating`, `timestamp`) schema and exceeding the 500K-row threshold by nearly $2\times$. Distributed processing is required because the user-item matrix is sparse and high-dimensional, ALS factor updates involve repeated joins that benefit from Spark's shuffle infrastructure, and sustained streaming throughput (we target 50–80 ev/s with bursts) outpaces single-process tooling. Data challenges include duplicate (`user`, `item`) pairs (we keep the latest by timestamp), out-of-range ratings (clipped to $[1, 5]$), and free-text fields that complicate CSV parsing (`multiLine=true`).

4 Machine Learning Component

4.1 Preprocessing

We drop rows with null `reviewerID`, `asin`, or `overall`, restrict ratings to $[1, 5]$, and deduplicate (`user`, `item`) pairs by keeping the latest `unixReviewTime`. `StringIndexer` converts `reviewerID` and `asin` into integer keys that ALS expects.

4.2 Training

We train ALS with `coldStartStrategy="drop"`, `nonnegative=True`, and an 80/20 random split (seed 42). The baseline configuration is `rank=10`, `regParam=0.1`, `maxIter=10`.

4.3 Evaluation and Tuning

RMSE is computed on the held-out 20%. If RMSE exceeds 1.5 we run a $2\times 2\times 2$ grid over `rank` $\in \{10, 20\}$, `regParam` $\in \{0.05, 0.1\}$, `maxIter` $\in \{10, 15\}$ and keep the best configuration.

In our run, the baseline configuration (`rank=10`, `regParam=0.1`, `maxIter=10`) produced **RMSE = 0.9001** on 196,333 held-out ratings, well below the 1.5 threshold, so no grid search was needed. Training fit time: ~ 26 s on `local[3]`.

5 Streaming Component

5.1 Kafka Setup

Kafka 2.12-3.7.0 runs in KRaft single-node mode from a portable folder (`$KAFKA_DIR/bin/kafka-server-start.sh`). The `interactions` topic uses **3 partitions** keyed by `user_id%3`; this preserves per-user event ordering while balancing across the three Spark cores. `recommendations` and `alerts` use 1 partition each since they are downstream sinks consumed by the dashboard.

5.2 Spark Structured Streaming

- **Source:** `kafka.subscribe = interactions`, `maxOffsetsPerTrigger = 2000`, `startingOffsets = latest`.
- **Parsing:** `from_json` with an explicit schema; rows whose `evt` struct is `null` are flagged malformed and dropped. A defensive `filter` also drops out-of-range ratings.
- **Watermark:** `event_time, 1 minute`.
- **Window:** 30-second size, 10-second slide, computed on `event_time`.

5.3 Window Analytics and Custom Metric

Per-window aggregates are: average rating per item, interaction count per user, and our custom **trending score**:

$$\text{trending}(i, w) = \text{count}(i, w) \cdot \overline{\text{rating}}(i, w) \cdot \exp\left(-\frac{\Delta t}{5}\right)$$

where Δt is the gap between the window end and the latest event for item i in minutes. The exponential decay rewards **recent** bursts within a window while still weighting items by quality and volume; a flat count-only metric would over-rank stale items that happened to peak earlier in a sliding window.

6 ML + Streaming Integration

At streaming-app start we read the saved ALS factor parquets into the driver process and use them for per-batch Top-5 generation. Each micro-batch:

1. Collects distinct user IDs.
2. Computes ALS scores via a NumPy dot product against the broadcast item-factor matrix.
3. Blends with the latest trending state: $\text{score} = 0.7 \cdot \widehat{\text{ALS}} + 0.3 \cdot \widehat{\text{trending}}$, both normalized to $[0, 1]$.
4. Writes the Top-5 to the `recs/` parquet sink and the `recommendations` Kafka topic.

Cold-start users (not seen in batch training) fall back to the current top-trending list.

7 Alert System and Late Data Handling

7.1 Alert Rules

- **Trending item:** `avg_rating > 4.5` AND `count ≥ 10` in the same window.
- **User burst:** a single user emits ≥ 20 events in one window (e.g. a bot or a power user spree).

Alerts go to console, the `output/alerts/` parquet sink, and the `alerts` Kafka topic.

7.2 Late Data

With `watermark = 1 minute` and `outputMode=append`, records arriving later than the watermark are silently dropped. We chose `append` (rather than `update`) because alert freshness and bounded state outweigh the benefit of re-emitting late updates.

8 Latency Measurement (Bonus)

The producer stamps `event_time_ms` at send time. The streaming app's `foreachBatch` sink subtracts that from the Kafka `ingestion` timestamp and writes per-record latency to `output/latency/`. Knobs applied to stay under 5s: `trigger=ProcessingTime("1 second")`, `maxOffsetsPerTrigger=500`, Kryo serializer, `spark.sql.shuffle.partitions=8`, factor matrices kept in driver memory (no per-batch I/O), and console alert sink dropped to reduce py4j overhead. Output sinks live on WSL-native ext4 (`~/mp3-output`) to avoid the `/mnt/c` 9p protocol bug that corrupts Spark state stores under sustained writes.

Measured over 3,352 samples in a 5-minute run:

Percentile	End-to-end latency (ms)
p50	1,202
p95	2,576
p99	3,256

p95 = 2,576 ms is well under the 5 s budget; the +1 latency bonus is claimed.

9 Results

Metric	Value
ALS RMSE (held-out 20%)	0.9001
Training rows	786,286
Test rows	196,333
Sustained producer rate	30 ev/s (synthetic)
Latency p50 / p95 / p99 (ms)	1,202 / 2,576 / 3,256
Latency samples collected	3,352
Recommendations rows generated	26,165
Unique users served Top-5 recs	5,034
Distinct items recommended	947
Alert windows triggered (5-min run)	39

The trending-item alert fired repeatedly on each producer-spiked `item_id` (e.g. 599, 39258, 60796) within one window of the spike beginning, with average ratings consistently above 4.85 and per-window event counts in the hundreds, confirming the trending detector reacts correctly to synthetic bursts.

10 Differentiation

Books domain (not the default movies); custom time-decayed trending score; hybrid recommender mixing ALS and trending with a normalized 70/30 blend; cold-start fallback to top-trending; per-user partition keying that aligns the 3 Kafka partitions with 3 Spark cores; sub-five-second latency budget measured per record.

11 Challenges and Lessons Learned

- Aligning Spark 4.1.1 (Scala 2.13) with the matching `spark-sql-kafka-0-10_2.13` connector -mismatched Scala builds silently fail at JSON parsing.
- Streaming state can grow unbounded; we cap the trending state map and rely on watermark-driven aggregation cleanup.
- ALS training is a one-shot batch job; integrating it with streaming requires either re-loading factors (chosen) or running predictions on executors with broadcast variables.

12 Reproducing the Pipeline

12.1 Environment

Run inside WSL2 (Ubuntu) with Kafka 2.12-3.7.0, Spark 4.1.1 (Scala 2.13.17), and OpenJDK 17. The Kafka folder is portable; no `apt` install.

12.2 One-time setup

```
sudo apt install -y python3-venv python3-full dos2unix
export KAFKA_DIR=~/.kafka_2.12-3.7.0
cd "Mini Project 3"
dos2unix infra/*.sh run_all.sh
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install streamlit-autorefresh
python src/download_dataset.py
```

12.3 Quick path one command

```
bash run_all.sh # then open http://localhost:8501
```

12.4 Manual path 5 terminals

Each terminal first activates the venv and exports KAFKA_DIR, MP3_OUTPUT_DIR=~/.mp3-output, MP3_DATA_DIR=~/.mp3-data.

```
# Terminal 1 broker
bash infra/start_kafka.sh
bash infra/create_topics.sh

# Terminal 2 batch ALS (once)
python src/train_als.py

# Terminal 3 producer
python src/kafka_producer.py --mode synthetic --rate 30

# Terminal 4 Spark streaming
spark-submit \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1 \
  --master "local[3]" --conf spark.driver.memory=6g \
  src/streaming_app.py

# Terminal 5 dashboard
streamlit run src/dashboard.py --server.address 0.0.0.0 --server.port 8501
```

12.5 Stop

```
kill $(cat ~/.mp3-output/logs/*.pid) 2>/dev/null
bash infra/stop_kafka.sh
pkill -f kafka.Kafka
```

12.6 Verification

```
# event count in interactions topic
"$KAFKA_DIR/bin/kafka-get-offsets.sh" \
  --bootstrap-server localhost:9092 --topic interactions

# latency snapshot
python -c "
```

```
import pandas as pd, glob, os
files = [f for f in glob.glob(os.path.expanduser(
    '~/mp3-output/latency/**/*.parquet'), recursive=True)
    if '_temporary' not in f and os.path.exists(f)]
df = pd.concat([pd.read_parquet(f) for f in files])
df = df[df.latency_ms.between(0, 600000)]
print('p50=%dms p95=%dms p99=%dms' % (
    df.latency_ms.quantile(0.5),
    df.latency_ms.quantile(0.95),
    df.latency_ms.quantile(0.99)))
"
```